

Lab 4 – VibeCoding

Chat History / Vibe Coding Record

Student	Samarth M H
Repository	SETAPESU26/37_climber
Personal Lab Repository	mhsamarth31/SE-LABS-PES1UG24CS407
Project	Climber – Python / Pygame
Assignment	Lab 4: VibeCoding

1. Assignment Workflow

The project was completed iteratively using Vibe Coding. The original game was first run and a before-video was recorded. The broken functionality and missing features were then addressed one task at a time, with testing after each change. Separate Git commits were maintained for the individual development tasks.

Task	Implementation	Status
1	Fix Height score so it stores the maximum height reached and does not decrease when the player falls.	Completed
2	Implement platform_color(index, total) so platform colors change progressively with platform index.	Completed
3	Implement moving_platform_speed(index, total) using the existing platform movement mechanism.	Completed
4	Implement on_coin_collected(coin, score) for the coin-collection effect without changing scoring/collision logic.	Completed

2. Task 1 – Fix Height Score

Goal: Make Height represent the maximum height reached during the current run.

Vibe Coding prompt used:

```
Analyze the existing game.py code for the Climber game. Task 1: Fix the Height score bug. The Height displayed in the HUD should represent the maximum height the player has reached during the current run. Once the player reaches a height, the Height score must never decrease if the player falls or moves downward. Identify the current code responsible for calculating/updating the Height score, explain why it decreases, and make the smallest possible code change. Do not modify unrelated game logic or implement the other README tasks yet.
```

Validation: The implementation was tested by climbing and then falling. The Height value remained at the highest value reached.

Commit: Fix height score tracking

3. Task 2 – Platform Colors

Goal: Implement platform_color(index, total) while preserving the existing rendering system.

Vibe Coding prompt used:

```
Implement Task 2 from the repository README: platform_color(index, total). Inspect how platform colors are determined and where the function is used. Implement the function according to the README, make the smallest necessary change, preserve gameplay and the Task 1 fix, and do not implement moving_platform_speed or on_coin_collected yet.
```

Debugging/validation: The function was verified to be called for all generated platforms. Stored RGB values were checked, and a temporary red-color test confirmed that the Pygame rendering path was working. The final implementation uses a visible green-to-purple progression based on platform index.

Commit: Implement platform colors

4. Task 3 – Moving Platform Speed

Goal: Implement `moving_platform_speed(index, total)` without rewriting the existing movement mechanism.

Vibe Coding prompt used:

```
Implement Task 3 from the repository README: moving_platform_speed(index, total). Inspect the README and existing game.py. The Platform class already contains the horizontal movement mechanism, including speed initialization, movement, and direction reversal. Implement only the missing decision function. Return the appropriate speed for platforms that should move and None or 0 for static platforms. Do not modify platform_color, on_coin_collected, scoring, player movement, collision detection, camera behavior, or unrelated logic.
```

Validation: The game was run after implementation to verify that the selected platforms use the existing horizontal oscillation mechanism.

Commit: Implement moving platform speed

5. Task 4 – Coin Collection Effect

Goal: Implement `on_coin_collected(coin, score)` while preserving existing coin scoring and collision logic.

Vibe Coding prompt used:

```
Implement Task 4 from the repository README: on_coin_collected(coin, score). Inspect the existing game.py. The game already detects coin collection, marks the coin as taken, adds 50 to coin_score, and calls on_coin_collected immediately afterward. Implement only the missing coin-collection effect. Do not change coin detection/scoring or unrelated gameplay. Use a simple reliable effect that does not require external asset files.
```

Validation: Coin collection was tested to ensure the coin disappears, the existing score behavior is preserved, and the collection effect executes without crashing the game.

Commit: Implement coin collection effect

6. Final Submission Structure

The completed work is intended to be placed in the existing personal Software Engineering lab repository under a Lab-4 folder:

```
SE-LABS-PES1UG24CS407/  
  Lab-4/  
    game.py  
    before-video.mp4  
    after-video.mp4  
    chat-history.pdf
```

7. Deliverables Checklist

Deliverable	Status
Before video (~10 seconds)	Completed
Updated Python/Pygame code	Completed
Task 1–4 implementations	Completed
After video (~10 seconds)	To be uploaded
Vibe Coding chat history PDF	This document

This document records the Vibe Coding workflow, prompts, implementation decisions, debugging, and validation carried out during the Lab 4 project.